

The Node.js Event Loop

Async Programming Fundamentals & Execution Order

Overview

This topic covers one of the most important concepts in Node.js — the **Event Loop**. Understanding the event loop is the key to writing correct asynchronous code. It explains why Node.js can handle thousands of operations without freezing, and it answers a question every Node.js developer faces: *"Why did this callback run before that one?"*

Concept	What you will learn
Event Loop Phases	The six phases, what runs in each, and why order matters
Microtasks vs Macrotasks	Two separate queues, their priorities, and how they interleave
<code>process.nextTick</code> & <code>setImmediate</code>	What they are, the difference, and when to use each
Blocking the Event Loop	How to identify and fix code that stalls Node.js

i Note

Prerequisites: You should already be comfortable running Node.js scripts from the terminal and understand what a function and a callback are. If you have not completed Topics 1 and 2 of this module, review them first.

1. The Big Picture — Why Does the Event Loop Exist?

1.1 Node.js is Single-Threaded

A **thread** is like a worker in an office. Traditional servers (e.g., Apache) create *one worker per request*. If 1,000 requests arrive simultaneously, 1,000 workers are created, which consumes a lot of memory and CPU.

Node.js takes a different approach: it uses **one single worker** (the main thread) for all JavaScript code. This sounds limiting, but combined with the event loop, it becomes very efficient.

Analogy

The Restaurant Analogy

Imagine a single chef (Node.js) in a kitchen. When a customer orders a slow dish that takes 20 minutes in the oven, the chef does *not* stand in front of the oven doing nothing. Instead, the chef puts the dish in the oven, sets a timer, and starts preparing other orders. When the timer rings, the chef comes back to finish that dish.

The **event loop** is the chef's internal loop — constantly checking timers, picking up finished tasks, and processing the next item on the list.

1.2 Blocking vs. Non-Blocking I/O

In server applications, most time is spent *waiting* — waiting for a database to respond, a file to be read, or a network request to return. This waiting is called **I/O (Input/Output)**.

- **Blocking I/O** — the program stops and waits until the I/O finishes. Nothing else can run.
- **Non-blocking I/O** — the program hands off the I/O work (to the OS or a thread pool managed by `libuv`), and continues running other code. When the I/O finishes, a *callback* is placed in the event loop queue to be executed.

Node.js uses non-blocking I/O by default for almost all operations. The event loop is the engine that makes this possible.

1.3 The Node.js Architecture (High Level)

Node.js is built on top of two major components:

- **V8** — Google's JavaScript engine that compiles and executes JS code. It also manages the *call stack* and the *heap* (memory).
- **libuv** — A C library that provides the event loop, the thread pool for heavy I/O, and OS-level async operations (timers, networking, file system).

i Note

When you write `fs.readFile()`, it is *not* V8 that reads the file — it is **libuv**, operating in the background on a separate thread (in the thread pool). When the read is complete, libuv places the callback in the event loop queue, and V8 picks it up and executes it.

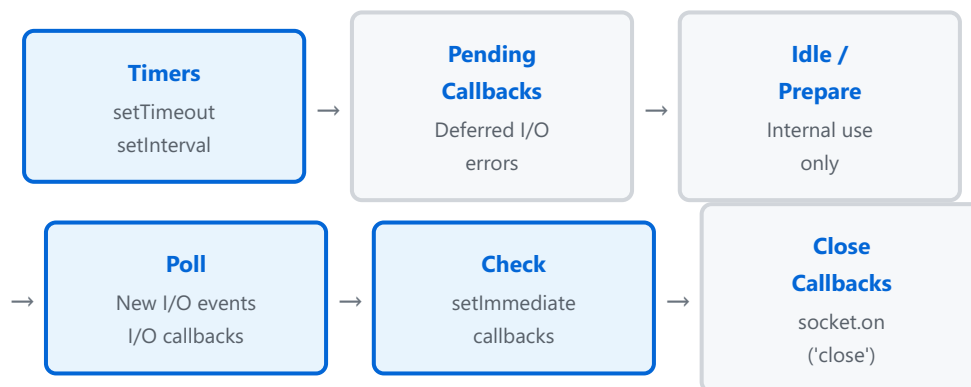
1.4 The Call Stack

The **call stack** is where JavaScript executes code. It follows a Last-In, First-Out (LIFO) principle — the last function pushed onto the stack is the first one executed and removed.

The event loop can only push a callback onto the call stack when it is **empty**. This is the fundamental rule.

2. The Event Loop Phases

The event loop is not a single queue. It is structured as a series of **phases**, and each phase has its own queue. When Node.js starts, it processes all synchronous code first, then enters the event loop and cycles through these phases repeatedly.



↻ The loop repeats until there are no more tasks

The **highlighted phases** are the ones you will work with directly in code. Let us examine each phase.

Phase 1 — Timers

This phase processes callbacks scheduled by `setTimeout()` and `setInterval()`. It does *not* run a timer the moment its delay expires. Instead, it runs the callback the next time the event loop reaches the *Timers phase* **after** the delay has elapsed.

i Note

A `setTimeout(fn, 0)` does **not** mean "run immediately." It means "run as soon as possible, but only after the current synchronous code finishes and after the event loop reaches the Timers phase." The minimum delay is actually ~1 ms in Node.js, and the actual execution time depends on OS scheduling and what else is in the queue.

Phase 2 — Pending Callbacks

Executes I/O callbacks that were deferred to the next iteration of the loop. For example, TCP errors from the previous iteration may be processed here. You do not schedule callbacks into this phase directly — it is managed internally by Node.js.

Phase 3 — Idle / Prepare

Used only internally by Node.js. You will never interact with this phase directly.

Phase 4 — Poll MOST IMPORTANT

The poll phase is the heart of the event loop. It does two things:

1. Retrieves new I/O events from the OS.
2. Executes their callbacks (e.g., data received from a network request, a file finished reading).

If the poll queue is empty and there is a `setImmediate()` callback waiting, the event loop moves immediately to the Check phase. Otherwise, it will wait here for new I/O events up to a calculated timeout.

Phase 5 — Check

This is where `setImmediate()` callbacks execute. The Check phase runs *immediately after* the Poll phase completes. This makes `setImmediate()` useful when you want a callback to run after the current I/O events but before any timers.

Phase 6 — Close Callbacks

If a socket or handle is closed abruptly (e.g., `socket.destroy()`), the `'close'` event callback is processed here. Without this phase, cleanup work would happen at an unpredictable time.

3. Microtasks vs. Macrotasks

Beyond the six phases, Node.js maintains two special queues that are processed **between** every phase transition. These are called *microtask queues*, and they have higher priority than any phase callback.

Microtask Queues (High Priority)

- `process.nextTick` queue — runs first
- `Promise` microtask queue — runs second

Processed between *every* phase transition.

Macrotask Queues (Phase Queues)

- Timers — `setTimeout`, `setInterval`
- I/O callbacks — file reads, network
- Check — `setImmediate`
- Close — socket close events

Processed inside their respective phase.

Analogy

The VIP Queue Analogy

Imagine a bank with two kinds of customers: regular customers (macrotasks) and VIP customers (microtasks). After serving each regular customer, the teller must first serve *all* waiting VIP customers before calling the next regular customer. The `process.nextTick` customers are even higher VIPs — they always go ahead of the Promise VIPs.

3.1 The Execution Order Rule

Every time a phase transition happens (e.g., Timers → Pending Callbacks), Node.js processes queues in this strict order:

- 1 All **synchronous code** on the call stack runs first.
- 2 All `process.nextTick` **callbacks** in the nextTick queue.
- 3 All **Promise callbacks** in the Promise microtask queue.
- 4 **Next phase macrotask** — one callback from the current event loop phase.
- 5 Go back to step 2, until all queues are empty.

i Note

In Node.js v11 and later, microtasks run between *each* macrotask callback, not just between phases. This makes it consistent with how browsers work. If you are on an older version, behaviour may differ.

4. Code Examples — Step by Step

The following examples build progressively. Each one introduces a new concept so that the final example makes complete sense.

Example 1 Synchronous Execution — The Call Stack in Action

Before adding any async code, understand how synchronous code executes. Functions are pushed onto the call stack one at a time and removed when they return. The event loop is not involved here at all.

```
// example1.js
console.log('Step 1 - Start');

function greet(name) {
  console.log('Step 2 - Hello, ' + name);
}

greet('Node.js');

console.log('Step 3 - End');
```

Output:

```
Step 1 - Start
Step 2 - Hello, Node.js
Step 3 - End
```

Nothing surprising here. Every line runs in the exact order it appears. The call stack is empty when the script finishes, and Node.js exits.

Example 2 Introducing Async — `setTimeout` with 0 ms Delay

Now we add `setTimeout`. Even with a delay of `0`, the callback does **not** run immediately. It is handed to libuv, and the Timers phase picks it up *after* the current synchronous code finishes.

```
// example2.js
console.log('A - synchronous (runs immediately)');

setTimeout(function onTimeout() {
  console.log('B - inside setTimeout (runs later)');
}, 0); // delay is 0 ms, but still async!

console.log('C - synchronous (runs immediately)');
```

Output:

A – synchronous (runs immediately)
C – synchronous (runs immediately)
B – inside `setTimeout` (runs later)

Why does C print before B? Because when Node.js encounters `setTimeout`, it registers the callback with libuv and moves on. The callback only runs after the call stack is empty and the event loop reaches the Timers phase.

Example 3 Introducing `process.nextTick` — The Highest Priority

`process.nextTick(callback)` schedules a callback to run *before* the event loop moves to its next phase — even before Promises. Think of it as saying: "finish what you are doing right now, then immediately run this before anything else."

```
// example3.js
console.log('A – synchronous');

setTimeout(() => {
  console.log('B – setTimeout (Timers phase)');
}, 0);

process.nextTick(() => {
  console.log('C – process.nextTick (runs before any phase)');
});

console.log('D – synchronous');
```

Output:

A – synchronous
D – synchronous
C – `process.nextTick` (runs before any phase)
B – `setTimeout` (Timers phase)

Notice that **C runs before B**. The `nextTick` queue is flushed completely before the event loop enters any phase (including the Timers phase where `setTimeout` lives).

Example 4 Adding Promises — The Second Microtask Queue

Promise callbacks (`.then()` , `.catch()`) are also microtasks, but they run *after* the `process.nextTick` queue is fully drained.

```
// example4.js
console.log('A - synchronous');

setTimeout(() => {
  console.log('B - setTimeout');
}, 0);

process.nextTick(() => {
  console.log('C - nextTick (priority 1 microtask)');
});

Promise.resolve().then(() => {
  console.log('D - Promise .then (priority 2 microtask)');
});

console.log('E - synchronous');
```

Output:

```
A - synchronous
E - synchronous
C - nextTick (priority 1 microtask)
D - Promise .then (priority 2 microtask)
B - setTimeout
```

The execution order confirms the rule:

Synchronous → nextTick → Promise microtasks → Event Loop phases

Example 5 Introducing `setImmediate` — The Check Phase

`setImmediate(callback)` schedules a callback in the *Check phase*, which runs right after the Poll phase. When compared to `setTimeout(fn, 0)` in the *top-level scope* (outside of any I/O callback), the order between the two is **non-deterministic** — it depends on the OS and system load.

```
// example5.js - top-level scope (order NOT guaranteed between setTimeout and setImmediate)
setTimeout(() => {
  console.log('setTimeout');
```

```
}, 0);

setImmediate(() => {
  console.log('setImmediate');
});

// You may see either order at the top level.
// Run this file several times to observe the non-determinism.
```

Output:

```
// Possible output 1:
setTimeout
setImmediate

// Possible output 2:
setImmediate
setTimeout
```

⚠ Warning

The order between `setTimeout(fn, 0)` and `setImmediate` is only non-deterministic at the *top level*. Inside an I/O callback, `setImmediate` **always** runs before any `setTimeout`. See the next example.

Example 6 setImmediate vs setTimeout Inside an I/O Callback

Inside an I/O callback, you are already in the Poll phase. After the Poll phase finishes, the event loop goes to Check (`setImmediate`) before cycling back to Timers (`setTimeout`). Therefore, `setImmediate` will **always** win here.

```
// example6.js
const fs = require('fs');

// Read a file – this callback runs in the Poll phase
fs.readFile(__filename, () => {
  console.log('A – I/O callback (Poll phase)');

  setTimeout(() => {
    console.log('B – setTimeout (next Timers phase)');
  }, 0);
});
```

```
setImmediate(() => {  
  console.log('C - setImmediate (Check phase, runs first)');  
});  
});
```

Output:

```
A - I/O callback (Poll phase)  
C - setImmediate (Check phase, runs first)  
B - setTimeout (next Timers phase)
```

This is guaranteed and deterministic. Inside an I/O callback, always prefer `setImmediate` when you want something to run "at the end of this I/O cycle."

Example 7 The Full Execution Order — Everything Together

This is the definitive example. Study the output carefully and verify it against the execution order rule.

```
// example7.js  
const fs = require('fs');  
  
console.log('1 - synchronous start');  
  
setTimeout(() => console.log('2 - setTimeout'), 0);  
  
setImmediate(() => console.log('3 - setImmediate'));  
  
process.nextTick(() => console.log('4 - nextTick (first microtask queue)'));  
  
Promise.resolve().then(() => console.log('5 - Promise (second microtask queue)'));  
  
fs.readFile(__filename, () => {  
  // This runs in the Poll phase  
  console.log('6 - fs.readFile I/O callback');  
  
  setTimeout(() => console.log('7 - setTimeout inside I/O'), 0);  
  
  setImmediate(() => console.log('8 - setImmediate inside I/O (wins over setTimeout)'));  
  
  process.nextTick(() => console.log('9 - nextTick inside I/O'));  
  
  Promise.resolve().then(() => console.log('10 - Promise inside I/O'));
```

```
});  
  
console.log('11 - synchronous end');
```

Output:

```
1 - synchronous start  
11 - synchronous end  
4 - nextTick (first microtask queue)  
5 - Promise (second microtask queue)  
2 - setTimeout (or 3, order may vary at top level)  
3 - setImmediate (or 2, order may vary at top level)  
6 - fs.readFile I/O callback  
9 - nextTick inside I/O  
10 - Promise inside I/O  
8 - setImmediate inside I/O (wins over setTimeout)  
7 - setTimeout inside I/O
```

The key observations:

- Lines 1 and 11 are always first — synchronous code runs before the event loop.
- Lines 4 and 5 are next — microtask queues drain before any event loop phase.
- Lines 2 and 3 are non-deterministic at the top level.
- Inside the I/O callback (lines 6–10), `setImmediate` (8) always beats `setTimeout` (7).
- Microtasks (9, 10) inside the I/O callback still drain before the next macrotask.

5. `process.nextTick` — Deep Dive

5.1 What It Is

`process.nextTick(callback)` places the callback at the front of the microtask queue. It runs after the current operation completes, but *before* the event loop moves forward at all.

5.2 Common Use Case — Ensuring Async Callback Consistency

A common problem in library design is functions that are sometimes synchronous and sometimes asynchronous. This inconsistency causes bugs. Using `process.nextTick` ensures a callback is always called asynchronously, even if the result is immediately available.

```
// BAD: inconsistent – sometimes sync, sometimes async
function readData(cache, callback) {
  if (cache) {
    callback(cache);          // synchronous!
  } else {
    fs.readFile('data.txt', (err, data) => {
      callback(data);         // asynchronous
    });
  }
}

// GOOD: always async using process.nextTick
function readDataFixed(cache, callback) {
  if (cache) {
    process.nextTick(() => callback(cache)); // forced async
  } else {
    fs.readFile('data.txt', (err, data) => {
      callback(data);
    });
  }
}
```

5.3 The Danger of Recursive nextTick

```
// DANGER – this will starve the event loop!
function infiniteNextTick() {
  process.nextTick(infiniteNextTick);
}
infiniteNextTick();
// I/O, timers, and everything else will NEVER run.
// Node.js will be stuck in the nextTick queue forever.
```

Output:

```
// No output – the program hangs.  
// I/O callbacks, setTimeout, and setImmediate never get a chance to run.
```

⚠ Warning

Because `process.nextTick` drains *completely* before moving forward, a recursive `nextTick` will starve the event loop. If you need to schedule recursive async work, use `setImmediate` instead — it yields control back to the event loop after each callback.

5.4 setImmediate as a Safe Alternative for Recursion

```
// SAFE – setImmediate gives the event loop a chance to breathe  
function processItems(items, index = 0) {  
  if (index >= items.length) return;  
  
  console.log('Processing item:', items[index]);  
  
  setImmediate(() => processItems(items, index + 1));  
  // Between each call, other I/O events and timers can run  
}  
  
processItems(['apple', 'banana', 'cherry']);
```

Output:

```
Processing item: apple  
Processing item: banana  
Processing item: cherry
```

6. Blocking the Event Loop

6.1 What "Blocking" Means

Since Node.js is single-threaded, if a piece of code runs for a long time *synchronously*, the call stack is occupied and the event loop cannot process *anything* else — no new requests, no timers, no I/O callbacks. This is called **blocking the event loop**.

Analogy

The Restaurant Analogy (revisited)

Blocking the event loop is like the chef suddenly deciding to hand-sharpen a knife for 5 minutes without stopping. During those 5 minutes, no new orders are taken, no dishes leave the kitchen, and all customers are left waiting.

6.2 Common Causes of Blocking

- Long-running loops or CPU-intensive computation (sorting huge arrays, cryptographic operations)
- Synchronous file operations inside request handlers: `fs.readFileSync` , `fs.writeFileSync`
- Synchronous parsing of large JSON objects: `JSON.parse` on very large strings
- Poorly-written, deeply-recursive synchronous functions

6.3 Blocking Example

```
// example_blocking.js
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/slow') {
    // This blocks the event loop for 5 seconds!
    const start = Date.now();
    while (Date.now() - start < 5000) {
      // busy-wait – nothing else can run during this time
    }
    res.end('Done (but I blocked everyone else for 5 seconds)');
  } else {
    res.end('Fast response');
  }
});
```

```
server.listen(3000, () => console.log('Server running on port 3000'));

// If you hit /slow with one browser tab,
// ALL other requests (including fast ones) will also be delayed 5 seconds.
// This is the core problem with blocking code in Node.js.
```

Output:

Server running on port 3000

```
// Request to /fast while /slow is processing:
// STALLED – waits 5 seconds before getting a response
```

6.4 Fixing Blocking Code

The solution depends on the type of blocking:

- **File I/O** — use async versions: `fs.readFile` instead of `fs.readFileSync`.
- **CPU-intensive work** — use Worker Threads (`worker_threads` module) or offload to a child process.
- **Large synchronous loops** — break the work into chunks with `setImmediate` so other events can run between chunks.

```
// example_nonblocking.js – chunk a large loop using setImmediate
function processLargeArray(array, callback) {
  const CHUNK_SIZE = 1000; // process 1000 items at a time
  let index = 0;

  function processChunk() {
    // Process a chunk of items synchronously
    const end = Math.min(index + CHUNK_SIZE, array.length);
    while (index < end) {
      // Simulate work on each item
      array[index] = array[index] * 2;
      index++;
    }

    if (index < array.length) {
      // Yield to the event loop, then continue with the next chunk
      setImmediate(processChunk);
    }
  }
}
```



```
    } else {  
      callback(array); // done  
    }  
  }  
}  
  
processChunk();  
}  
  
// Generate a large array  
const bigArray = Array.from({ length: 10000 }, (_, i) => i + 1);  
  
console.log('Starting large computation (non-blocking)...');  
  
processLargeArray(bigArray, (result) => {  
  console.log('Done. First 5 results:', result.slice(0, 5));  
});  
  
console.log('This prints immediately – event loop is not blocked!');
```

Output:

```
Starting large computation (non-blocking)...  
This prints immediately – event loop is not blocked!  
Done. First 5 results: [ 2, 4, 6, 8, 10 ]
```

The key insight is that `setImmediate` between chunks allows the event loop to process incoming requests or I/O callbacks between each chunk, keeping the server responsive.

7. Quick Reference — Comparison Table

Function	Queue / Phase	Priority	Use Case
<code>process.nextTick()</code>	nextTick queue (microtask)	Highest	Ensure callback runs before I/O, defer within current operation
		High (after nextTick)	

Function	Queue / Phase	Priority	Use Case
<code>Promise.resolve().then()</code>	Promise microtask queue		Standard async/await, chained async operations
<code>setImmediate()</code>	Check phase (macrotask)	Medium (after I/O)	Run after current I/O, safe for recursion, chunked processing
<code>setTimeout(fn, 0)</code>	Timers phase (macrotask)	Low	Delay code by at least one event loop cycle
<code>setInterval(fn, n)</code>	Timers phase (macrotask)	Low	Repeat a callback every <code>n</code> milliseconds

✓ Best Practice

Rule of thumb: Use `setImmediate` instead of `setTimeout(fn, 0)` when you want to defer work to after the current I/O cycle. Use `process.nextTick` sparingly, only when you need the highest possible priority before any I/O.

8. Complete Execution Order — Summary

Here is the definitive mental model. For any given moment in a running Node.js program:

#	What runs	Type	Example
1	Current synchronous code on the call stack	Synchronous	<code>console.log()</code> , variable assignments, loops
2	All <code>process.nextTick</code> callbacks (drain completely)	Microtask	<code>process.nextTick(fn)</code>
3		Microtask	

#	What runs	Type	Example
	All Promise microtask callbacks (drain completely)		<code>Promise.resolve().then(fn)</code>
4	Next callback from the current event loop phase	Macrotask	<code>setTimeout</code> , I/O, <code>setImmediate</code>
5	Back to step 2 (drain nextTick again)	Microtask	After every macrotask

Interview Questions

The following questions are commonly asked in Node.js developer interviews. Understanding these thoroughly will help you answer questions about asynchronous JavaScript and Node.js internals with confidence.

Q1 — CONCEPTUAL

What is the Node.js event loop, and why does Node.js use it instead of creating a new thread for every request?

Q2 — EXECUTION ORDER

What will the following code print, and why?

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
process.nextTick(() => console.log('4'));
console.log('5');
```

Q3 — COMPARISON

What is the difference between `process.nextTick()` and `setImmediate()` ? When would you choose one over the other?

Q4 — PHASES

In which phase of the event loop does `setImmediate()` run? What phase runs before it, and what phase runs after it?

Q5 — PROBLEM SOLVING

What does "blocking the event loop" mean? Give two examples of code that blocks the event loop, and explain how you would fix each one.

Q6 — MICROTASKS

What is the difference between a microtask and a macrotask? Name two examples of each.

Q7 — TRICKY

A colleague uses `setTimeout(fn, 0)` inside an I/O callback to schedule cleanup. You suggest using `setImmediate` instead. What is your reason?

Q8 — ADVANCED

If you call `process.nextTick` recursively (i.e., inside its own callback), what happens to the event loop? How does this differ from using `setImmediate` recursively?

Practical Exercise

Task — Predict and Verify Execution Order

Part A: Predict the Output

Before running the code below, write down what you think the output will be, in order, with a reason for each line.

```
// exercise.js
const fs = require('fs');

console.log('start');

setTimeout(() => {
  console.log('timeout 1');
  process.nextTick(() => console.log('nextTick inside timeout'));
}, 0);

setImmediate(() => {
  console.log('immediate 1');
});

Promise.resolve()
  .then(() => {
    console.log('promise 1');
    return Promise.resolve();
  })
  .then(() => console.log('promise 2'));

process.nextTick(() => {
  console.log('nextTick 1');
  process.nextTick(() => console.log('nextTick 2 (nested)'));
});

fs.readFile(__filename, () => {
  console.log('readFile callback');
  setTimeout(() => console.log('timeout inside readFile'), 0);
  setImmediate(() => console.log('immediate inside readFile'));
});

console.log('end');
```

Part B: Run and Compare

Run the file with `node exercise.js` and compare the actual output with your prediction. For every line where you were wrong, write a sentence explaining *why* it ran in that position.

Part C: Modify and Extend

1. Add a second `process.nextTick` inside `promise 1`'s `.then`. Does it run before or after `promise 2`? Why?
2. Simulate a blocking operation inside the `fs.readFile` callback using a `while` loop that runs for 2 seconds. Observe how it affects the execution of other callbacks.
3. Refactor the blocking loop from Part C-2 using `setImmediate` to process in chunks without blocking.

Module 4 · Node.js & Express · Topic 3 of 11 · Next topic: *Modules, CommonJS, and ES Modules*

